**B.Tech Project Report: Smart Phishing URL Detection**

**Name:** Krishna Rai

**Reg No:** 25BAI11298

**Branch:** CSE (AI/ML)

---

**1. Project Overview**

This project is centered on building a security tool that doesn't just rely on old databases of "bad" links, but actually looks at the URL itself to spot danger. Phishing is getting smarter every day, so I wanted to create something that could analyze a link's structure and flag it as a threat instantly.

**2. The Problem at Hand**

Most people think a simple "blocklist" is enough. The problem is that hackers generate thousands of new URLs every hour. By the time a link is reported and blocked, the damage is already done. I set out to build a tool that uses Machine Learning to predict if a link is dangerous before anyone even reports it.

**3. Project Scope**

The scope of this work involves building a complete pipeline—from handling raw, messy URL data to deploying a working web dashboard. It’s designed to be a fast, local tool that any user can run on their machine to verify a suspicious link.

**4. Why Traditional Methods Fail**

Standard antivirus software often misses "zero-day" phishing links because they look for specific files or signatures. My approach is different; it looks at the "intent" of the URL by analyzing things like unusual subdomains and token frequency.

**5. Dataset Choice**

I used a massive dataset of over 500k URLs. Finding a balanced dataset was tough because there are way more "good" links on the internet than "bad" ones. I had to ensure the model saw enough examples of phishing to actually learn the red flags.

**6. Managing the 30MB File (LZMA)**

One big hurdle was GitHub's file size limit. To fix this, I used **LZMA (.xz) compression**. It’s much more efficient than a standard ZIP file. It allowed me to shrink the 30MB dataset down to just 7MB, making the whole repo much lighter and easier to clone.

**7. Cleaning the Data**

URLs are noisy. I wrote a custom tokenizer to strip out the "http://" and "https://" parts since they don't really help with classification. I then split the remaining string into tokens based on dots, slashes, and dashes to see the hidden patterns.

**8. Turning Text into Numbers (TF-IDF)**

Since computers can't read text, I used **TF-IDF**. This is a smart way to weigh words. It ignores common stuff like "www" but gives a high score to suspicious tokens like "secure-login" or "account-verify" that frequently show up in phishing attacks.

**9. The Choice of Logistic Regression**

I chose Logistic Regression because it’s incredibly fast. In security, you can't wait 10 seconds for a Deep Learning model to load. This model gives an answer in milliseconds, which is exactly what you need when you're browsing the web.

**10. Training vs. Testing**

I split the data using an **80/20 ratio**. I spent a lot of time making sure the 20% "test set" was a fair representation of real-world links so that the accuracy scores actually mean something.

**11. Technical Stack**

The project is built entirely in **Python**. I used scikit-learn for the ML heavy lifting and pandas for the data processing. The web part is handled by Streamlit because it's great for showing data visualizations.

**12. Designing the CLI**

I wanted a tool that felt natural for developers. The CLI has two modes: you can either pass a single URL for a quick check or enter an "interactive" mode to scan a whole list of links one by one.

**13. Building the Web App**

The Streamlit app is the "face" of the project. I used caching so that the model only has to train once when you first open the page. This makes the user experience much smoother since the predictions become instant.

**14. Understanding Accuracy**

While the accuracy is high, I learned that accuracy isn't everything. A model could be 99% accurate but still miss the most dangerous phishing links. That's why I focused heavily on the "recall" rate.

**15. The Importance of the Confusion Matrix**

This is where I spent most of my time during the evaluation. The confusion matrix shows exactly where the model gets confused—like when a safe site looks a bit too much like a phishing one.

**16. Training Results (Terminal)**

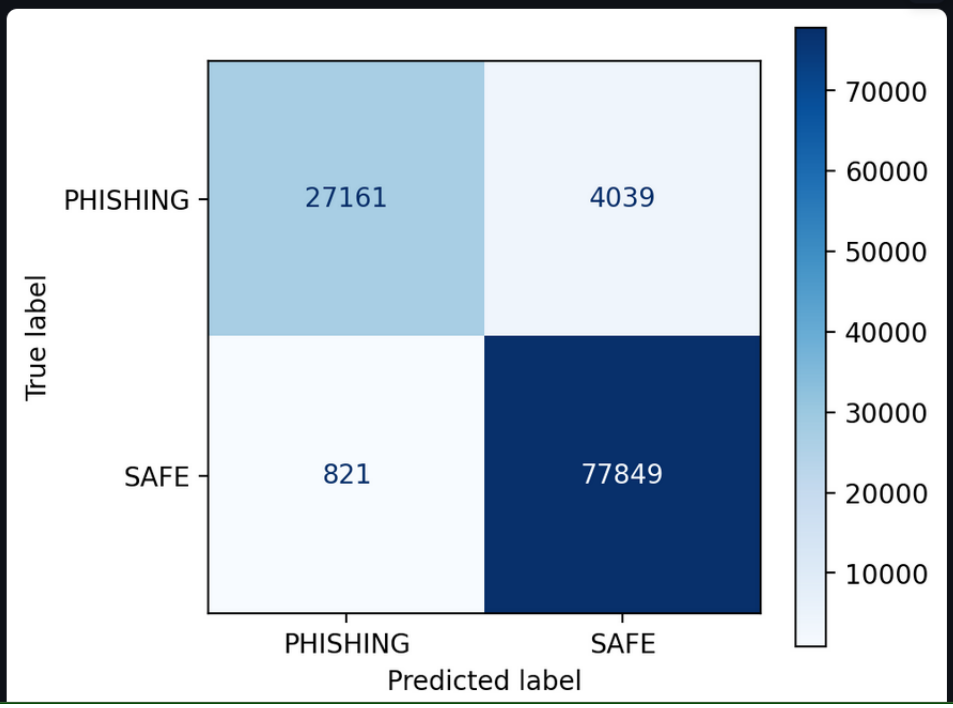
```
--------------------------------------------------
 URL Threat Classification Engine v1.0
 Uses TF-IDF & Logistic Regression
--------------------------------------------------

[INFO] Ingesting dataset: phishing_site_urls.csv...
[INFO] Performing train/test split (80/20)...
[INFO] Extracting features via TF-IDF Vectorization... (This may
take a moment)
[INFO] Fitting Logistic Regression classifier...
[INFO] Validating model accuracy...
[SUCCESS] Model trained. Validation Accuracy: 95.58%

[INFO] Confusion matrix — rows: true label, columns: predicted
          PHISHING   SAFE
PHISHING     27161   4039
SAFE           821  77849

--- Interactive Analysis Mode ---
Enter a URL to analyze. Type 'exit' to terminate.
```

**17. The Confusion Matrix Heatmap**

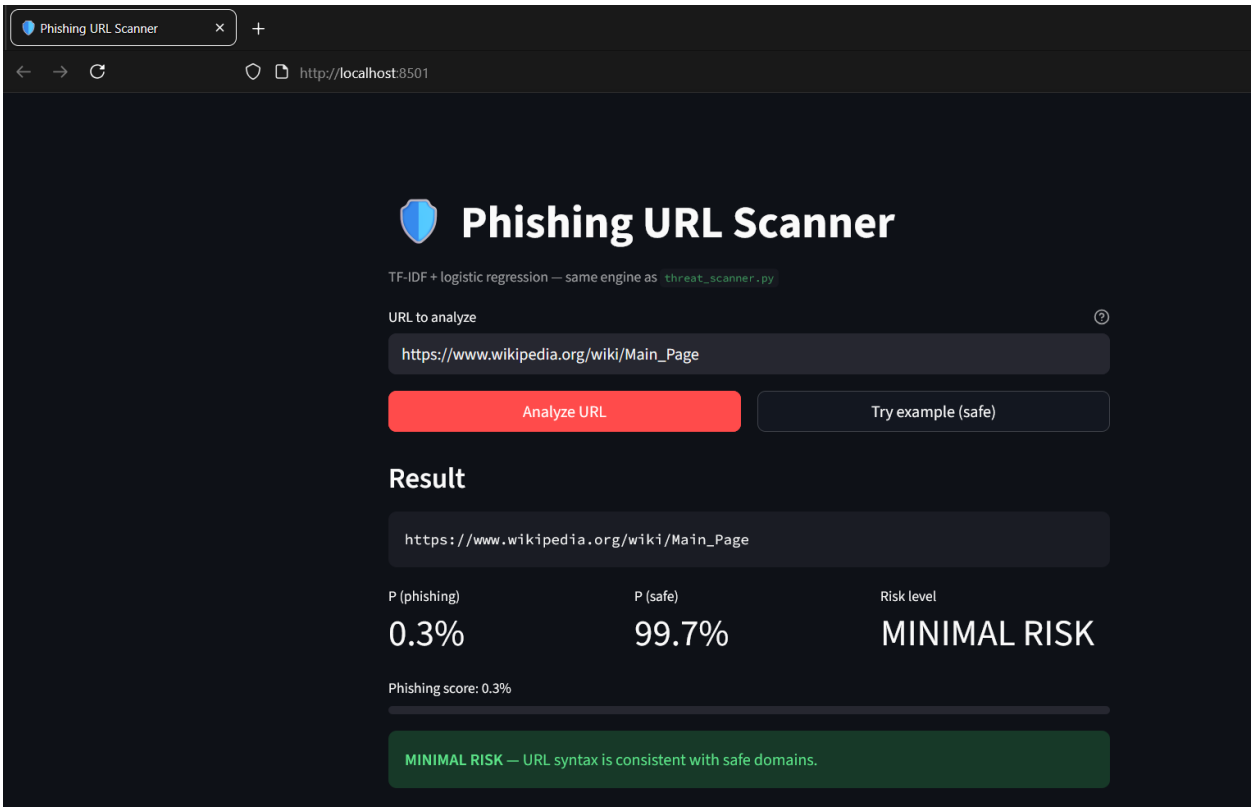
TF-IDF + logistic regression — same engine as `threat_scanner.py`

## Hold-out confusion matrix

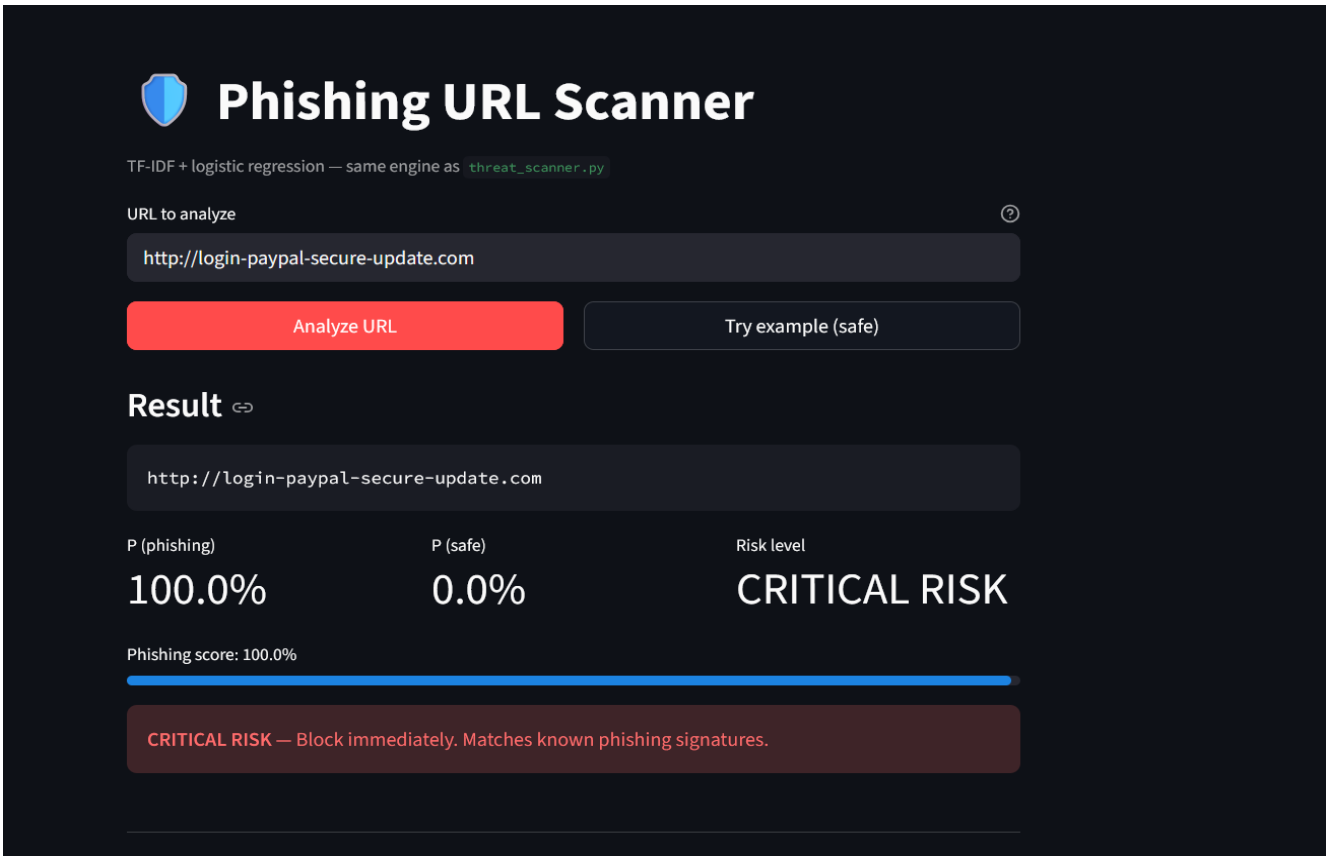
20% test split (same seed as CLI). Rows: actual label · Columns: predicted label.



## 18. Testing: A Safe Example



### 19. Testing: A Phishing Example




### 20. Closing Thoughts

This project was a great way to see how AI can be applied to real-world cybersecurity. While it's not perfect, it's a solid start. For future versions, I'd like to add a feature that checks the age of the domain to make the predictions even more accurate.

---

**Date:** March 28, 2026

**Candidate Signature:** Krishna Rai